ejercicio1.pl

```
%Ejercico 1
%abuelo(X, manuel) es X:= juan

padre(juan, carlos).
padre(juan, luis).
padre(carlos, daniel).
padre(carlos, diego).

padre(luis, pablo).
padre(luis, manuel).
padre(luis, ramiro).
abuelo(X,Y) :- padre(X,Z), padre(Z,Y).

hijo(X.Y) :- padre(Y,X).
hermano(X,Y) :- padre(Z,X) , padre(Z,Y).

%descendiente(X,Y) :- padre(Y, X).
%descendiente(X, Y) :- padre(Z, X) ,  descendiente(Z, Y).

descendiente(X,Y) :- padre(Y, X) ; padre(Z, X) , descendiente(Z, Y).

%CINCO, padre(X, pablo) , padre(X, Y).  esto incluiriai a pablo igula

ancestro(X, X).
ancestro(X, Y) :- ancestro(Z, Y), padre(X, Z).

%Se cuelga porque hace una busqeda infinita pero on hay mas resultados definiidos.

%ancestro2(X, Y) :- descendiente(Y,X).

ancestro2(X, Y) :- padre(X, Y).
ancestro2(X, Y):- padre(Z,Y), ancestro2(X, Z).

%ejercicio3
% menorOIgual(0, X) entra en el primer caso y busca infinitamente el succ del succ, porque aca nunca se ejecuta la otra rama??
%stack infinito, se hacen infinitos llamados o ramas con infinitos llamados.

natural(0).
natural(suc(X)) :- natural(X).

menorOIgual(X,X).
menorOIgual(X, succ(Y)):- menorOIgual(X, Y).

%ejercicio4

juntar([], L2, L2).
juntar([X|Xs] , L2, [X|Ys]) :-juntar(Xs, L2, Ys).

%ejercicio5

ultimo(L, U) :- append(_, [U], L).
```

```
%al reves. ifnitino
reverso([] ,[]).
reverso([X|Ls], Rs):- append(Z, [X] ,Rs) , reverso(Ls, Z).

prefijo(P, L) :- append(P, _, L).

sufijo(S, L) :- append(_, S, L).

sublista(S, L) :- sufijo(S, L), not(prefijo(S,L)).
sublista(S, L) :- prefijo(S, L).
sublista(S, [X|Xs]) :- sublista(S, Xs), not(sufijo(S,[X|Xs])) , not(prefijo(S, [X|Xs])). %evitar not


qcyo(S, L) :- append(_,R, L ),  append(S, _, R). %tira repetidas , rsat bien

pertenece(X, L) :- append(_ , [X, _] , L).



%ejercicio6
aplanarMal([],[]).
aplanarMal([X|Xs], [X|Ys]) :- aplanar(Xs,Ys). %el problema de esto es q va a dar true si tengo dos listas [[1,2],3], [[1,2],3]
aplanarMal([[]|Xs], Ys) :- aplanar(Xs,Ys).
aplanarMal([X|Xs],Ys):- append(X, Z, Ys) , aplanar(Xs,Z).



%EsLista
esListaVacia([]).

esLista([]).
esLista([_|T]):- esLista(T).

aplanar([],[]).
aplanar([X|Xs], Ys):- esLista(X) ,aplanar(X, Xaplanada), aplanar(Xs, Z) ,append(Xaplanada,Z,Ys).
aplanar([X|Xs], Ys):- not(esLista(X)) ,aplanar(Xs, As), append([X], As, Ys).



%interseccion, yo quiero q si el elemento aparece en
%interseccion([], _ , []).
%interseccion([X|Xs], [X|Ys], [X|Zs]):- interseccion(Xs, Ys, Zs), not(member(X,Zs)).
%interseccion([X|Xs], [Y|Ys], [X|Zs]):- Y \= X, member(X, Ys),interseccion(Xs, [Y|Ys], Zs), not(member(X,Zs)).
%interseccion([X|Xs], [Y|Ys], Zs):- Y \= X, not(member(X, Ys)),interseccion(Xs, [Y|Ys], Zs), not(member(X,Zs)).



interseccion([], _ , []).
```

```
interseccion([X|Xs], Ys, Zs) :- not(member(X, Ys)), interseccion(Xs, Ys, Zs).
interseccion([X|Xs], Ys, [X|Zs]):- member(X,Ys), interseccion(Xs, Ys, Zs).

size([], 0).
size([_|T], N) :-  size(T, Z), N is Z+1.  %Z is N-1 , size(T, Z)

partir(N, L, L1, L2) :- append(L1, L2, L), size(L, C), C > N ,Z is C-N ,size(L2, Z).

partir2(0, L, [] , L).
partir2(N, [X|Xs], [X|Ys],L2) :- N>0 ,Z is N-1, partir2(Z, Xs, Ys, L2).


borrar([],_ ,[]).
borrar([X|Xs] , X, L) :- borrar(Xs, X, L).
borrar([Y|Xs], X, [Y|Ys] ):- borrar(Xs, X, Ys), X \= Y.


sacarDUplicados([], []).
sacarDUplicados([X|Xs], [X|ResultadoTail]) :- borrar(Xs,X,SinX), sacarDUplicados(SinX,ResultadoTail).

borrarXUnaVez(X, [X|Xs] ,Xs).
borrarXUnaVez(X, [Y|Xs] ,[Y|Z]):- X \= Y, borrarXUnaVez(X, Xs, Z).


permutacion([],[]).
permutacion([X|Xs],Ys):- permutacion(Xs,R), append(L1, L2, R), append(L1,[X|L2], Ys).

reparto([], 0, []).
reparto(L, N, [Xs|Xss]):- N>0 ,append(Xs,A, L), N2 is N-1 ,reparto(A, N2,Xss).

repartoSinVacias([], []).
repartoSinVacias(L, [Xs|Xss]):- append(Xs, A, L), length(Xs,V), V>0, repartoSinVacias(A,Xss).

%LA IDEA ES Q TENGO EL 1, Y EL [2,3], EN TODAS LAS POSIBLES COMBINACIONES, METO AL 1 EN EL MEDIO, PERO TAMBIEN L1 PUEDE SER EL PRINCIPIO (LO ESTARIA METIENDO AL FINAL)
%reparto(L,N, LL) :- .

%ejercico 9.

% Con un Y mayor a x, SI NO, se v a a infinita.

desde(X,X).
desde(X,Y) :- N is X+1, desde(N,Y).

desdeReversible(X, Y):- nonvar(Y), Y >= X.
desdeReversible(X, Y):- var(Y), desde(X,Y).

%11
max(V1,V2,V1):- V1 >= V2.
```

```
max(V1,V2, V2):- V2 > V1.

vacio(nil).

raiz(bin(_, V, _), V).

altura(nil, 0).
altura(bin(I,_, D), N) :- altura(I,V1), altura(D, V2) ,max(V1,V2,Z), N is Z+1.

cantidadDeNodos(nil,0).
cantidadDeNodos(bin(I,_,D), N):- cantidadDeNodos(I, V1), cantidadDeNodos(D, V2), N is (1 + V1 + V2).

%inorder(+AB, -Lista)
inorder(nil, []).
inorder(bin(I,R,D), Lista):-inorder(I, LI), inorder(D, LD), append(LI, [R] ,Z), append(Z, LD, Lista).

arbolConInorder([], nil).
arbolConInorder(L, bin(I, R, D)):- append(Li, [R|Ld], L), arbolConInorder( Li, I), arbolConInorder(Ld, D).

menorA(nil, _).
menorA(bin(I,R,D), V):- R<V, menorA(I,V), menorA(D,V).

mayorA(nil, _).
mayorA(bin(I,R,D), V):- R>V, mayorA(I,V), mayorA(D,V).


abb(nil).
abb(bin(I, R ,D)) :- menorA(I, R), mayorA(D,R), abb(I), abb(D).

aBBInsertar(X,T1,T2):- inorder(T1, L), append(A,B,L), append(A,[X|B],L2), arbolConInorder(L2,T2), abb(T2).

coprimos(X,Y):- desde(2,N), paresQueSuman(X,Y,N), gcd(X,Y) =:= 1.

paresQueSuman(X,Y,N):- between(1,N, X), Y is N -X.

%14, yo quiero verificar que todas sumen N

%tengo q tener N listas en L, tienen que tener todas tamaño N

%+N , -L


%tengo el problema q si pongo size primero se van a generar infinitamente


% estoy tratando al x como si lo tuviese instanciado y no lo tengo, tenog un problema para llenar con ceros
sumanIconSizeN(0,0,[]).
sumanIconSizeN(N,I,[X|Xs]):- between(0,I,X), N > 0, I >= 0 ,N2 is N-1, I2 is I-X, sumanIconSizeN(N2,I2, Xs).
```

```
%LOGICA DE PRIMER ORDEN TIENE PREDICADOS, FUNCIONES VARIAB

sumanIconSizeNTodos(0,_,_,[]).
sumanIconSizeNTodos(K,N,I,[L|Ls]):- K > 0, sumanIconSizeN(N,I,L),K1 is K-1, sumanIconSizeNTodos(K1,N,I,Ls).

cuadradoSemiMagico2(N, L):- desde(0,I) ,sumanIconSizeNTodos(N,N,I,L).

filaIgualAPrimeraColumna([],[]).
filaIgualAPrimeraColumna([X|Xs], [[X|_]|Ls]):- filaIgualAPrimeraColumna(Xs, Ls).

sinPrimeraColumna([[]],[]).
sinPrimeraColumna([[_|Xs]|Es], [Xs|Ys]):- sinPrimeraColumna(Es,Ys ).

%- |
traspuesta([],[]).
traspuesta([L|Ls],O):- filaIgualAPrimeraColumna(L, O), sinPrimeraColumna(O, OsinColumna), traspuesta(Ls, OsinColumna).

cuadradoMagico(N,L):- cuadradoSemiMagico2(N,L), traspuesta(L,M), cuadradoSemiMagico2(N,M).


%17, un Y tal que cumpla P y no cumpla Q.

%El Y no afectaria el P porque no sale del not. Asi que solo buscaria el de P si es que existe algun Q q no cumpla el predicado.

% P(Y), not(P(X), Y != X)



%18
dif(V1,V2,V):- V is V1 - V2, V>= 0.
dif(V1,V2,V):- V is V2- V1, V > 0.

suma([], 0).
suma([X|Xs], V):- suma(Xs, Z), V is X + Z.

diferenciaSuma(L1,L2,V3):- suma(L1, V1), suma(L2,V2), V3 is abs(V1 - V2).

corteMasParejo(L,L1,L2):- append(L1,L2,L), diferenciaSuma(L1,L2,V1), not((append(L3,L4,L), diferenciaSuma(L3,L4,V2), V2<V1)).

%numero poderoso

primo(1):-!.
primo(2):-!.
primo(X):- Z2 is X-1, between(2,Z2,Z1), not(X mod Z1 =:= 0).

susPrimosLoSuperDividen(1):-!.
susPrimosLoSuperDividen(Y):-Y2 is Y-1, between(2,Y2,X2), primo(X2), Y mod X2 =:= 0,X22 is X2*X2, Y mod X22 =:= 0, Z3 is Y / X22 ,susPrimosLoSuperDividen(Z3), !.

```

```
proximoNumPoderoso(X,Y):- X1 is X+1 ,desde(X1,Y), susPrimosLoSuperDividen(Y), !.
```